



Computational thinking and AI
计算思维与人工智能
兴趣班

以诺十一学校 中国科学院计算所
龙溪实验中学

<https://ai-class-shiyilongyue.github.io/>

十、计算机如何认识图像

中国科学院计算技术研究所

邢云冰

助教：张蔚

2026.05.27

授课老师简介



邢云冰

中国科学院计算所
高级工程师

研究兴趣包括：数字人手语、人机交互与数字内容生成。

研究成果：

长期深耕人工智能模型架构设计、工程化实现与应用落地，在手语合成、人机交互及数字内容生成领域拥有深厚的技术积淀。

主导研发的冬奥手语播报数字人系统、气象节目的手语字幕系统、爱心小屋远程亲情互动系统、残疾人远程手语翻译服务平台、科教卫同屏互动服务平台等广泛应用于残疾人信息无障碍公共服务领域，打通了AI算法模型从实验室原型到规模化应用的闭环。

已积累发明专利30余项（已授权17项，已转让5项）。

获奖荣誉：

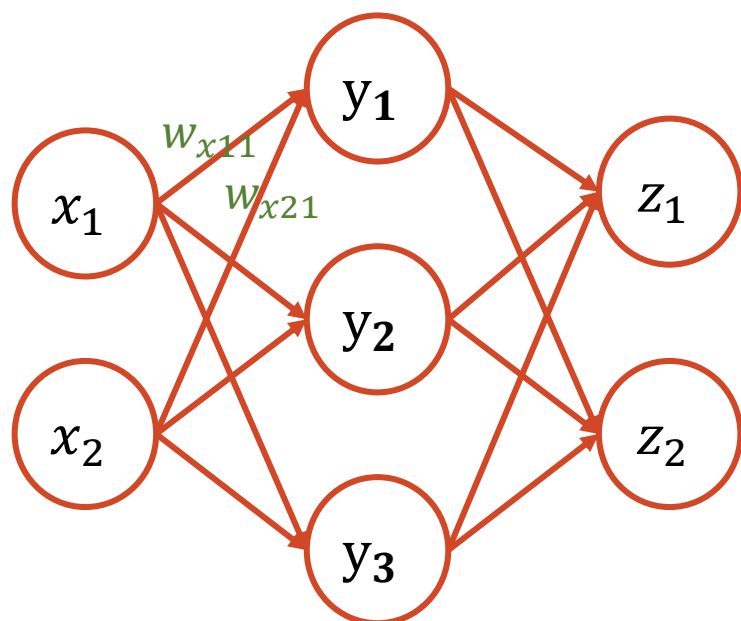
CCF科学技术发明一等奖

北京市科学技术奖二等奖（两次）

中国产学研合作创新成果奖

回顾：多层感知机

将前一层的输出作为后一层感知机的输入



$$y_1 = h(x_1 * w_{x11} + x_2 * w_{x21} + b_{y1})$$

$$y_2 = h(x_1 * w_{x12} + x_2 * w_{x22} + b_{y2})$$

$$y_3 = h(x_1 * w_{x13} + x_2 * w_{x23} + b_{y3})$$

$$h(x) = \begin{cases} 0 & \text{if } x \leq \theta \\ 1 & \text{if } x > \theta \end{cases}$$

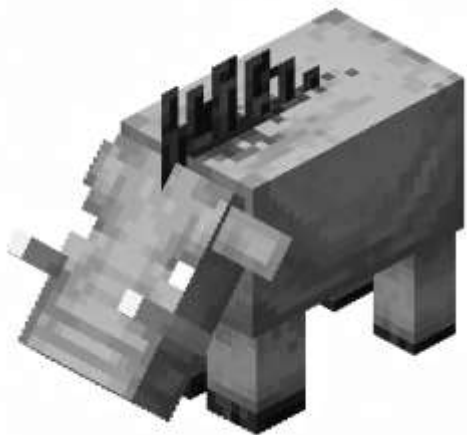
$$z_1 = h(y_1 * w_{y11} + y_2 * w_{y21} + y_3 * w_{y31} + b_{z1})$$

$$z_2 = h(y_1 * w_{y12} + y_2 * w_{y22} + y_3 * w_{y32} + b_{z2})$$

彩色图像：我的世界



灰度图像：我的世界



计算机认识图像：像素



彩色图像 (RGB)

由红、绿、蓝三原色混合而成，最接近真实世界

3通道



灰度图像 (Gray)

只有黑白灰，没有颜色，常用于文字识别

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

1通道



黑白图像 (Binary)

非黑即白，简单纯粹，适合处理轮廓和形状

1通道(0/1)

图像的清晰度

图像在水平方向上的
像素个数

width (宽)

图像在垂直方向上的
像素个数

height (高)

图像的颜色层数。彩
色通常为 3 (RGB)，灰
度为 1 (Gray)

channel (通道)

分辨率 = width × height

像素越多，画面越细腻

马赛克



身边的分辨率

常见分辨率清单

4000x3000 手机照片 (1200万像素)

1920x1080 电脑屏幕 (Full HD)

3840x2160 电视 (4K超高清)

28 x 28 手写数字

身边的分辨率

动手试一试

1

打开你的手机相册

2

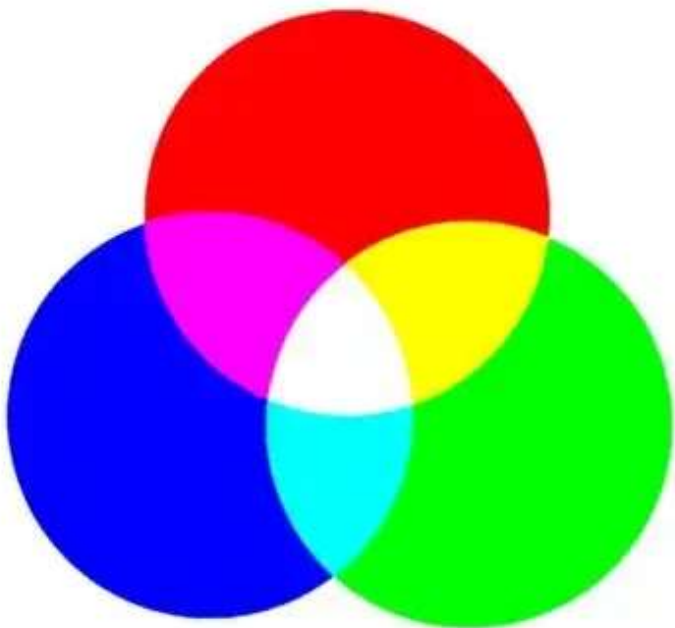
选择一张照片，点击“详细信息”或“i”图标

3

看看它的分辨率是多少？



微观原理：RGB三原色



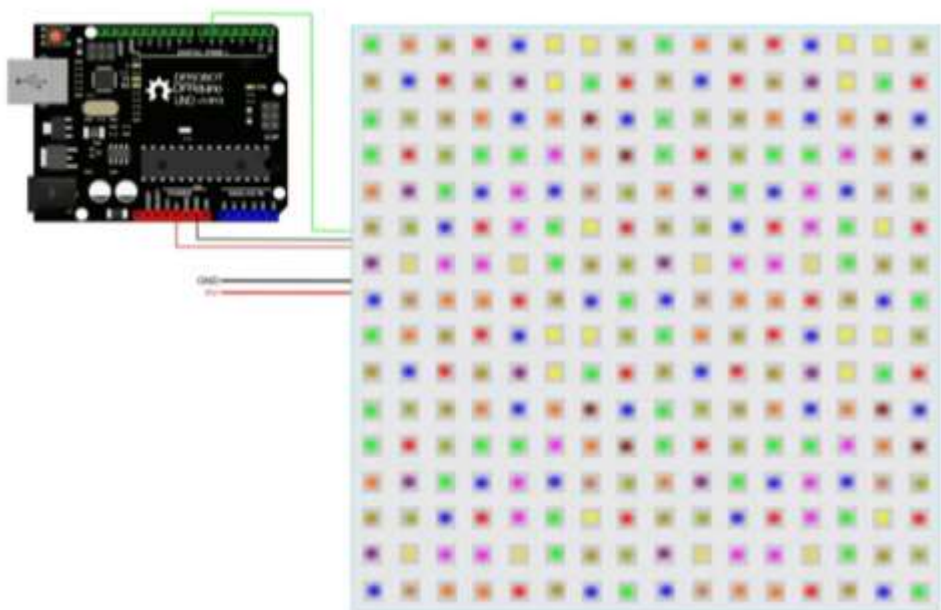
RGB三原色

RGB 三原色原理

每个 LED 灯珠内部包含红(R)、绿(G)、蓝(B)三个发光单元，通过不同亮度的组合，可以调配出任意颜色

- 1 全亮：对应数值 255 (uchar8)
- 0 全暗：对应数值 0 (uchar8)

现场演示：LED点阵

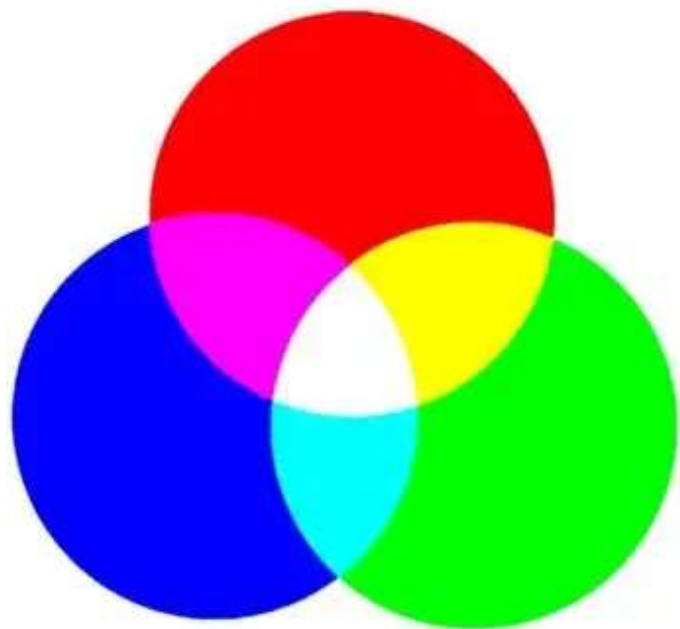


实物 16x16 点阵

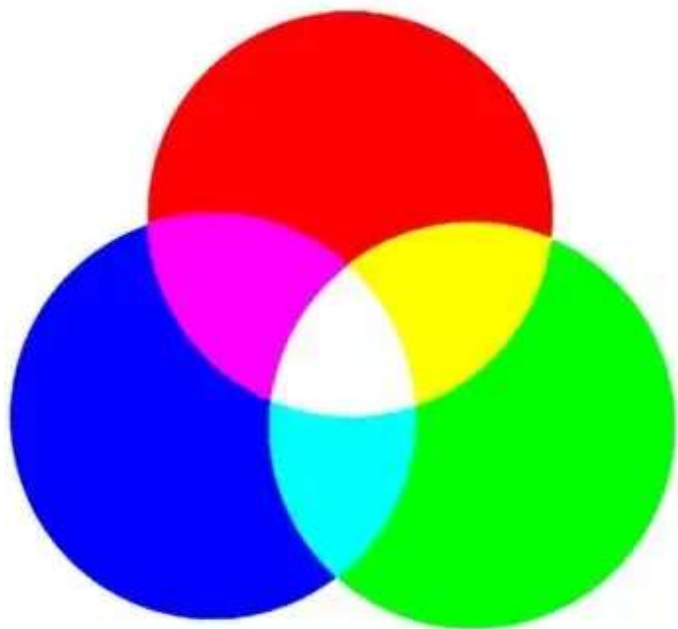
RGB 三原色原理

每个 LED 灯珠内部包含红(R)、绿(G)、蓝(B)三个发光单元，通过不同亮度的组合，可以调配出任意颜色

课堂提问



彩虹中的“五颜六色”也是这个原理吗



彩虹中的“五颜六色”也是这个原理吗

彩色打印机的基础颜色是什么

图像的存储和类型

像素排列与存储

图像是由无数个小方格（像素）组成的。计算机按照特定的顺序来记忆这些像素：

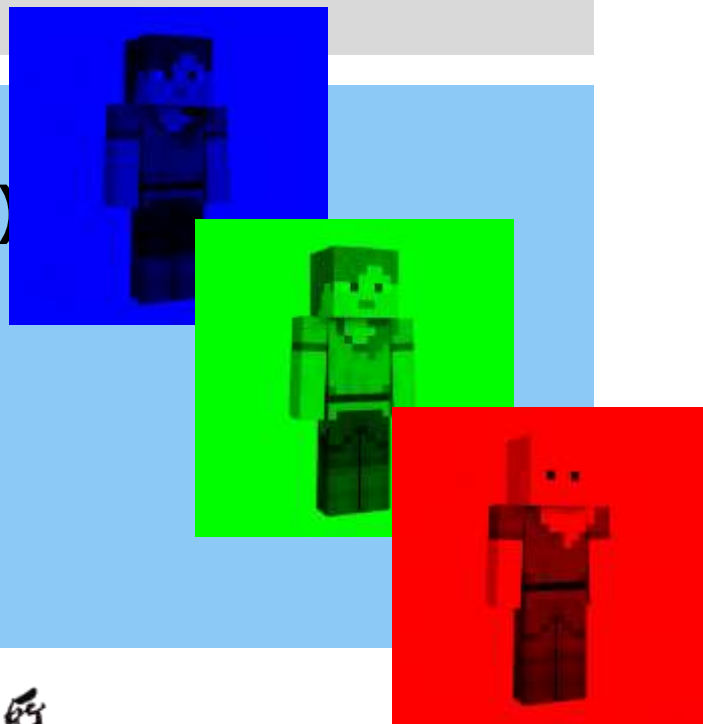
通道 (channel)



宽 (width)



高 (height)



数据类型

uint8

0-255 的整数，最常用的存储方式。节省空间，适合显示

float32

小数，更精确。适合AI 模型进行复杂的数学计算。对于图像，**0.0-1.0**。

归一化 (Normalization)

为了方便计算，我们通常将亮度映射到 0 到 1 之间

手写数字数据集：MNIST



手写数字在计算机眼里是什么？

每一个方格（像素）都有一个对应的数值。当我们写字时，笔画覆盖的区域数值会发生变化。

区域	状态	大约数值（归一化）
背景 (空白)	全暗	0.0 - 0.1
前景 (笔画)	全亮	0.9 - 1.0
边缘 (过渡)	半亮	0.4 - 0.6

如何让计算机认识图像



任务本质

输入：一张图像（由 0.0 – 1.0 组成的像素矩阵）

输出：一个类别标签（0 到 N 中的某一个数字）

计算机需要从杂乱的像素中找到**规律**

多层感知机识别图像

初始做法

为了让多层感知机（MLP）处理图像，我们必须将二维的像素矩阵“拉直”成一维的长向量。

[0.1 0.2 0.3]

[0.4 0.5 0.6]

[0.7 0.8 0.9]

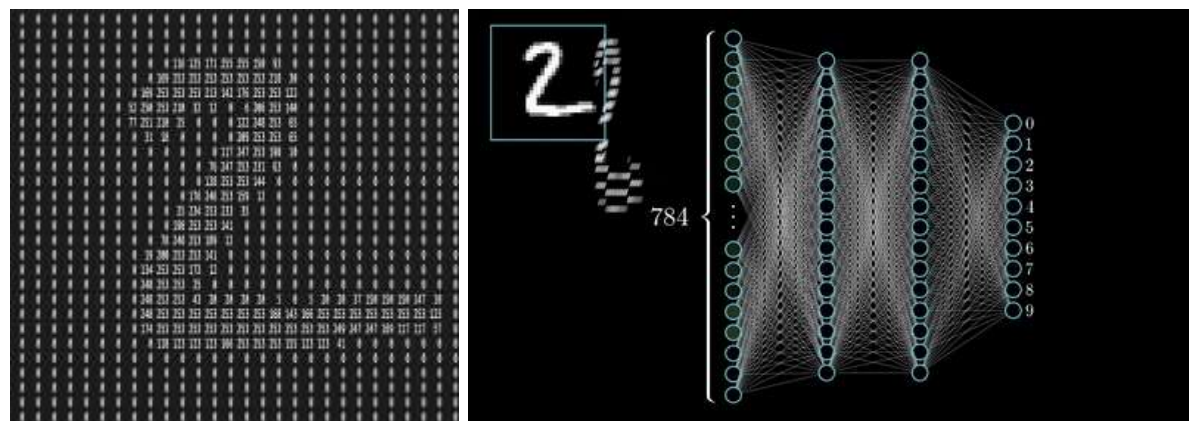


[0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]

致命的弱点

空间信息丢失：一维化后的数据只保留了左右邻居的信息，却彻底丢失了上下邻居的关联。

特征识别困难：图像中的关键特征（如圆圈、横杠）本质上是二维的空间结构。MLP 很难在这种“线型”数据中找回原本的形状。



神经网络的春天

天时：大数据技术成熟

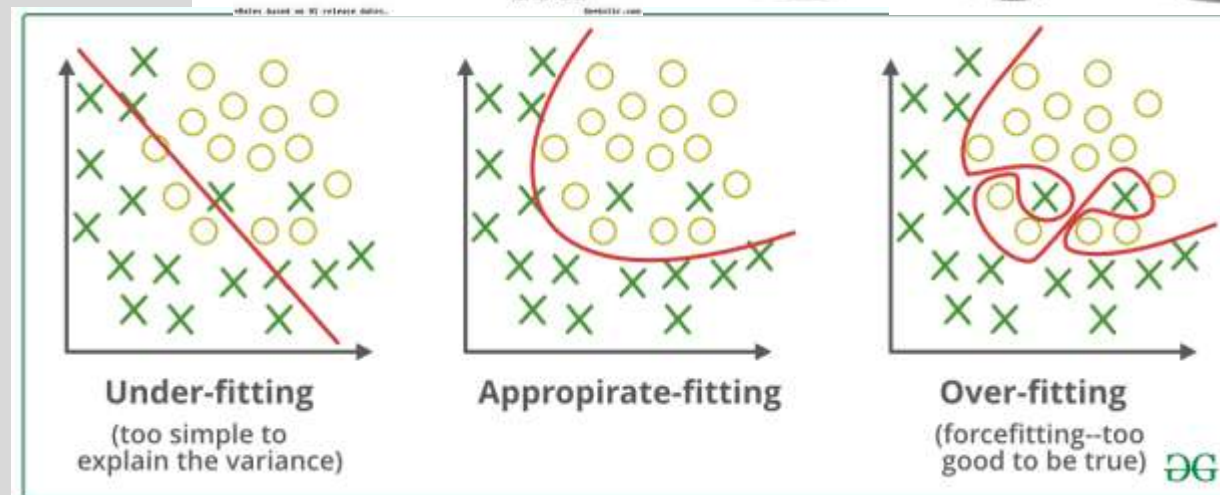
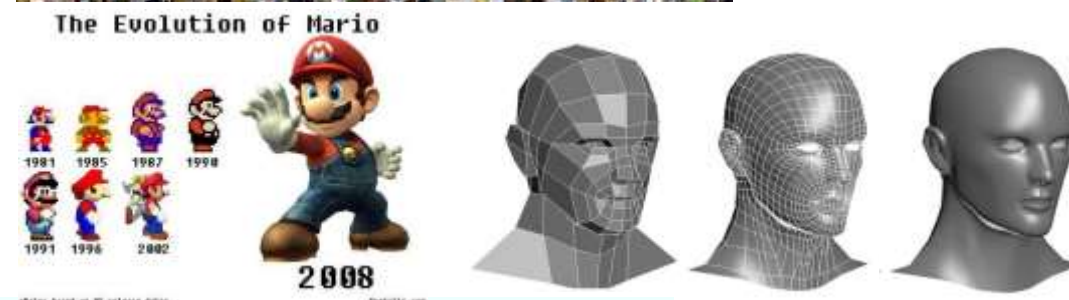
- 包含大量训练数据的数据集容易找到一般规律

地利：算力提升

- 游戏业的发展推动了GPU性能提升
- 使用GPU进行大规模并行训练

人和：模型训练技术

- 解决梯度消失
Relu激活、残差连接、BatchNorm等
- 解决过拟合
dropout、early-stop等



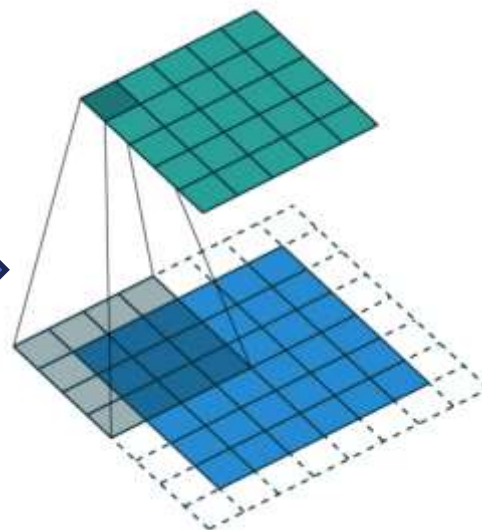
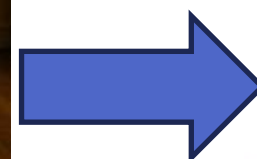
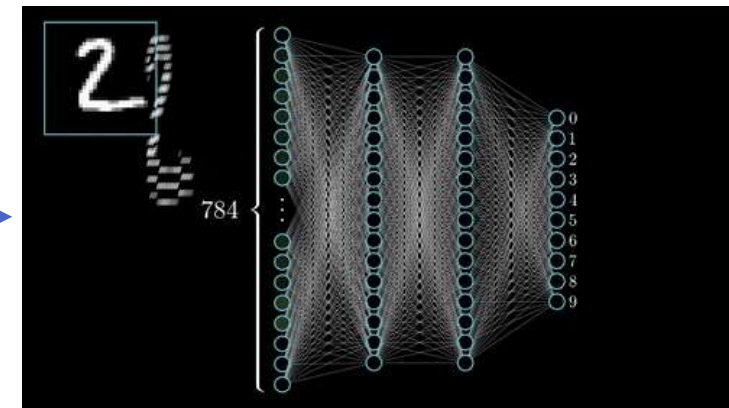
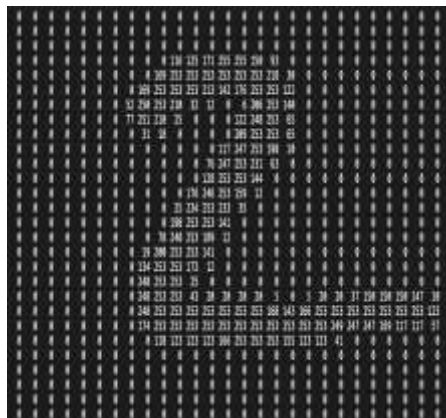
借鉴人类的视觉系统：卷积神经网络

直接使用层数加深的多层感知机

参数量太大
没有利用图像数据的空间关系

卷积核 (Kernel)

就像是一个“小窗口”或“印章”，在图像上滑动，每次只看一小块区域。



保留空间位置信息的方法：卷积+池化

卷积：算特征

两个二维矩阵，对应位置上的数字相乘，然后求和。这个过程能捕捉到图像的局部特征（如边缘、线条）。



-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2

0	1	1
0	1	0
1	-1	1

Kernel Channel #3

308

+

-498

+

164

+

1 = -25

Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

池化：选特征

二维矩阵分块取值

1	8	0	3
7	1	3	3
4	1	9	4
6	0	1	0



逐层卷积和重要算子

从简单到复杂

底层卷积：识别简单的边缘、线条和斑点

中层卷积：组合线条，识别出圆圈、拐角等局部形状

高层卷积：理解全局结构，识别出完整的数字特征

每一层都在对图像进行更高层级的抽象！

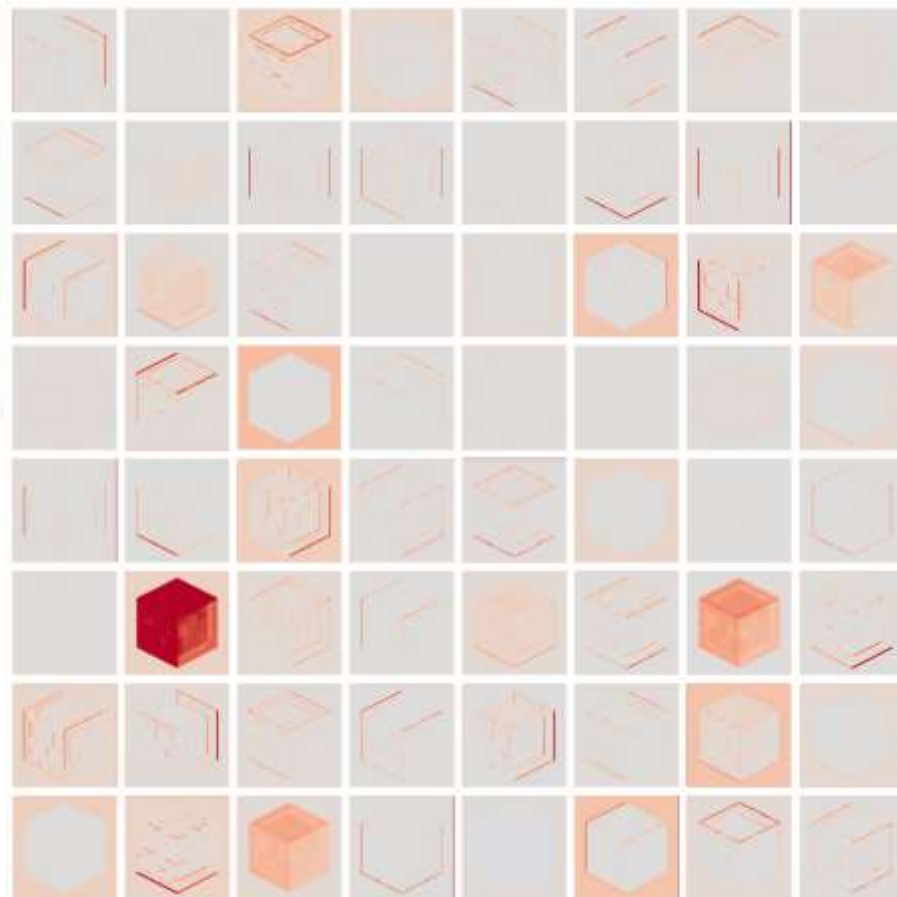
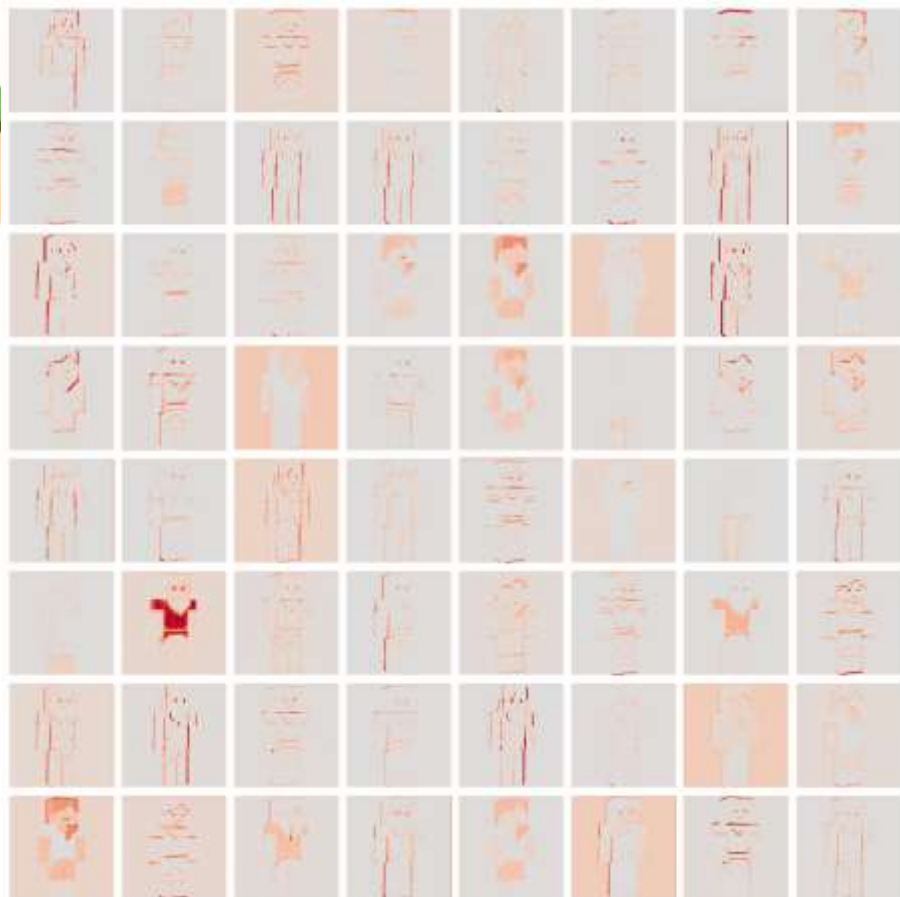
$$\text{out_size} = \text{in_size} + 2 * \text{padding} - (\text{kernel_size} - 1)$$

算子	作用	重要参数
Conv2d	特征提取	kernel_size/padding
Pool2d	下采样	kernel_size
Dropout	防止过拟合	随机断连
Linear	全连接	
ReLU	非线性映射	
Softmax	概率计算	

底层提取特征



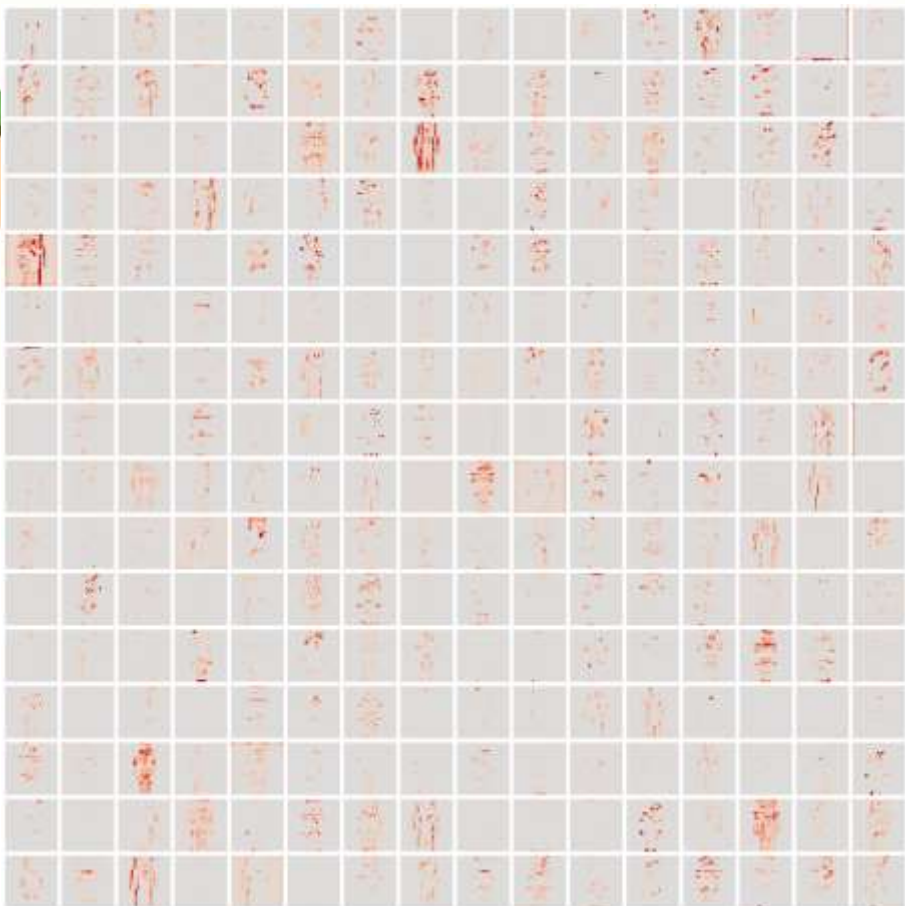
Computational thinking and AI
计算思维与人工智能
兴趣班



中层提取特征



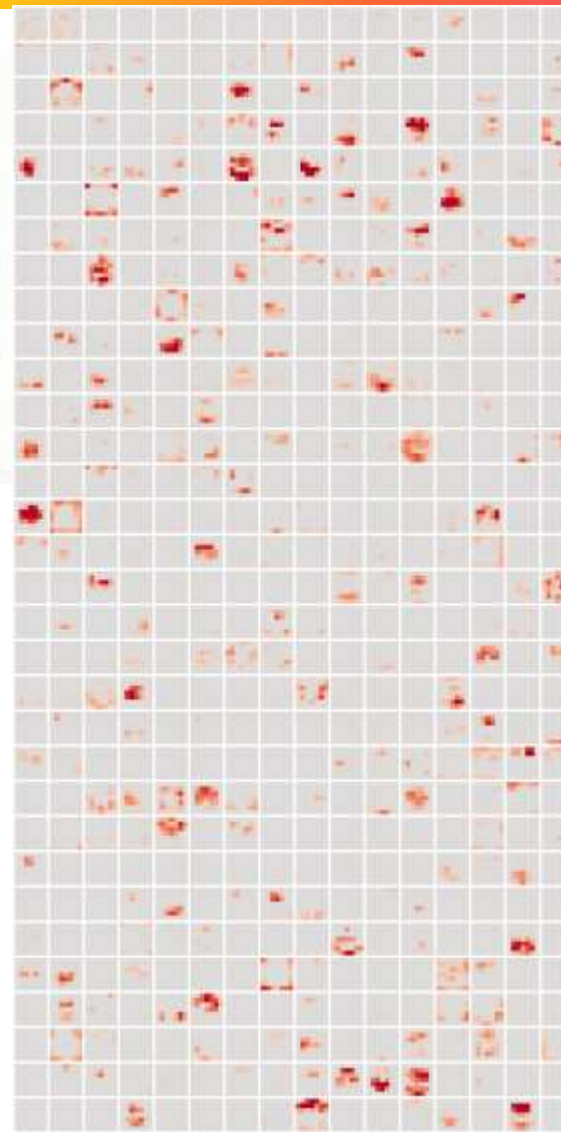
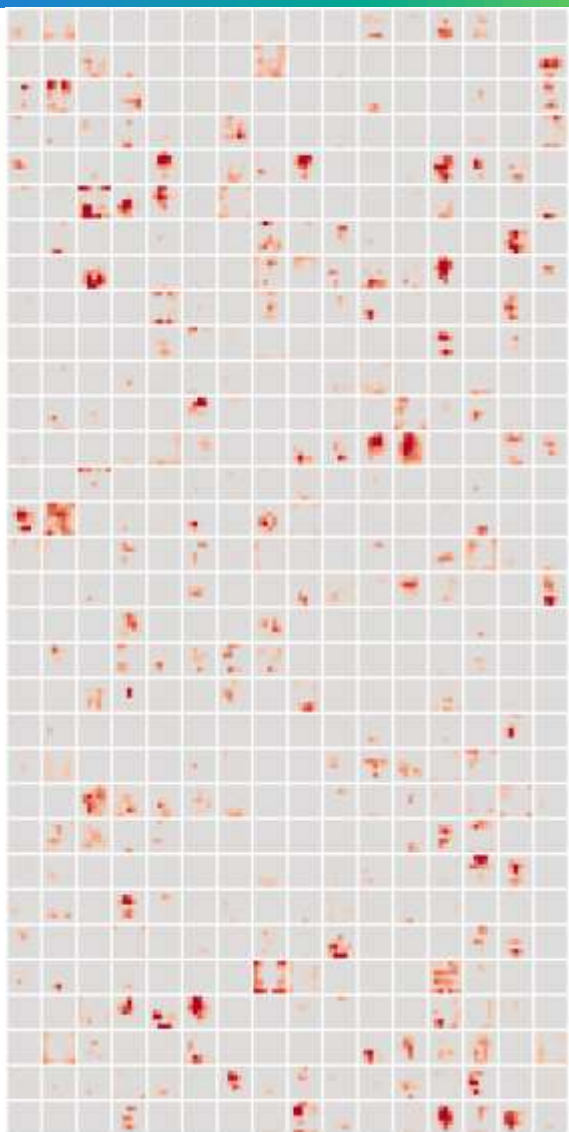
Computational thinking and AI
计算思维与人工智能
兴趣班



高层融合特征



Computational thinking and AI
计算思维与人工智能
兴趣班



卷积神经网络的一般结构

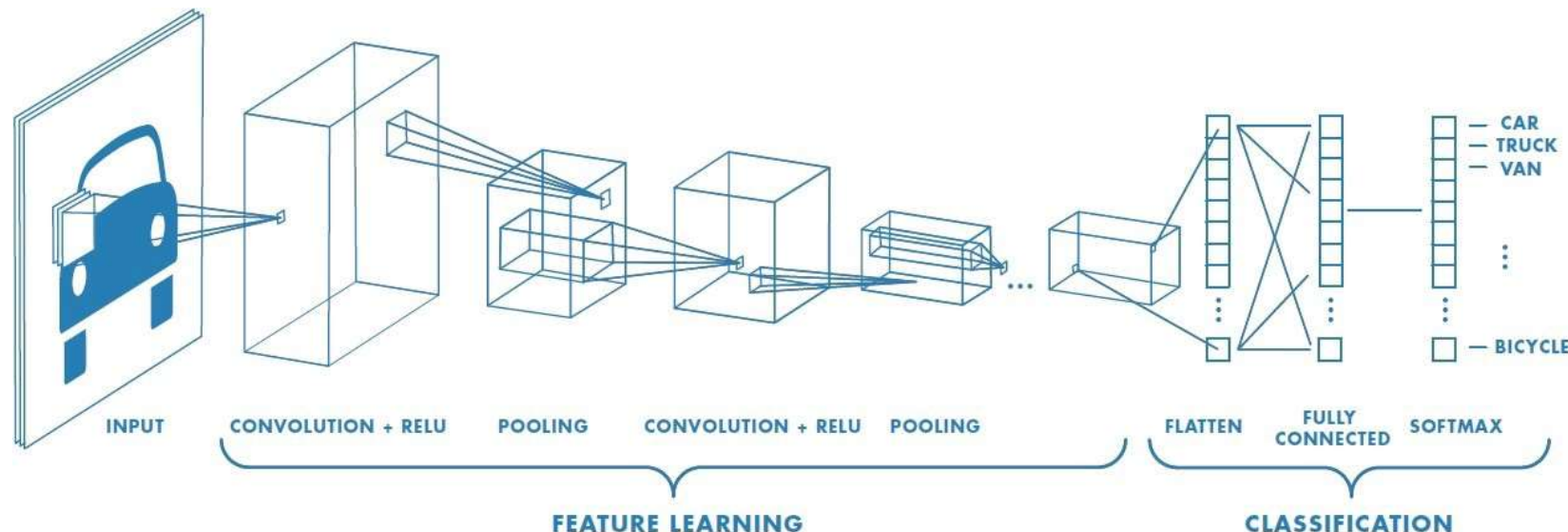
用于图像分类的CNN一般由特征提取部分和分类部分组成

特征提取部分

- 卷积层
- 激活函数
- 池化层

分类部分

- 全连接层/线性层
- Softmax层



卷积的数学本质

$$y = \text{relu}(x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \cdots + b)$$

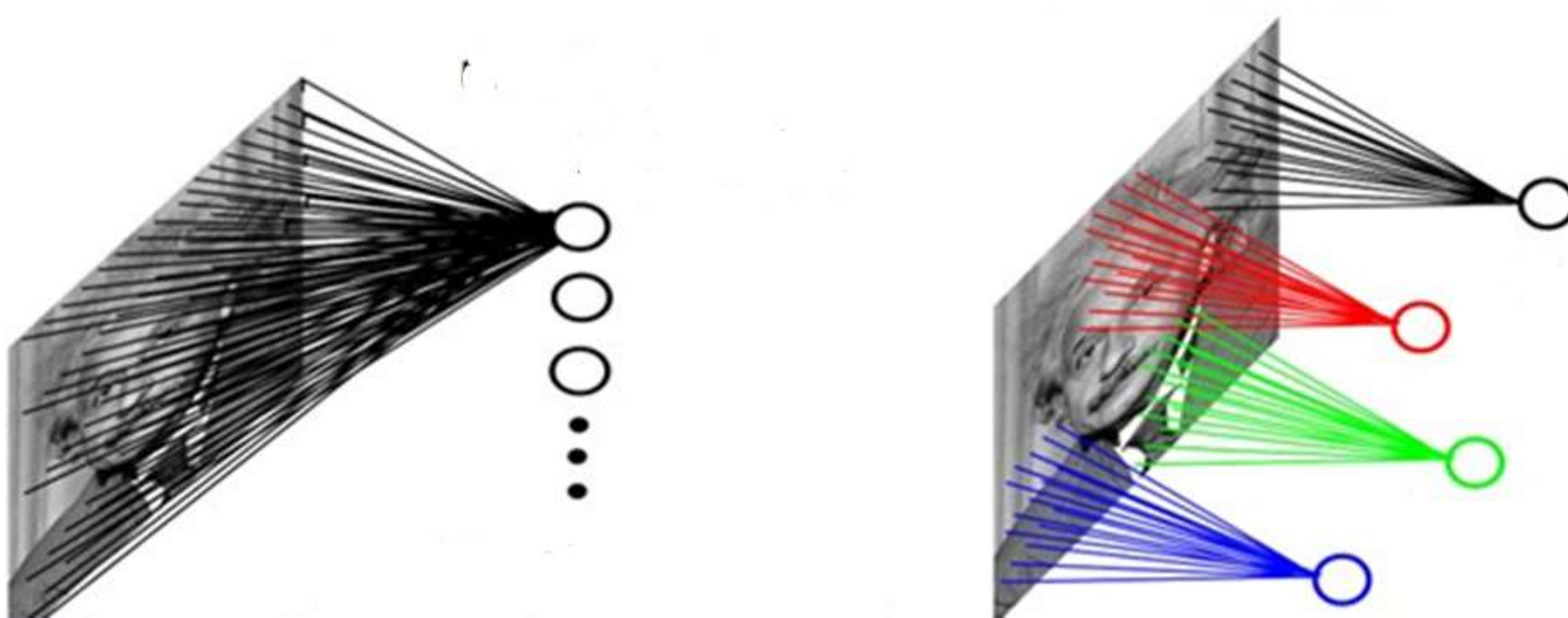
卷积计算 与 多层感知机 在形式上是高度统一的

组成部分	在CNN中的含义	在 MLP 中的含义
权重 w	卷积核 (Kernel) 的参数	全连接层的权重矩阵
偏置 b	每个卷积核对应的偏置项	每个神经元的偏置项
输入 x	二维图像的局部像素区域	一维向量中的每个元素/特征
参数 w 的数据量	$\text{kernel_size} * \text{kernel_size} * \text{in_channels} * \text{out_channels}$	$\text{in_features} * \text{out_features}$
激活函数	让神经网络具备处理复杂问题的能力	
训练目标	寻找最优的 w 和 b ，使模型预测结果最接近真实标签。	

卷积的退化：边界条件

$\text{kernel_size} = 1$

$\text{kernel_size} = \text{image_size}$



回顾：梯度下降法

1、前向传播（试错）

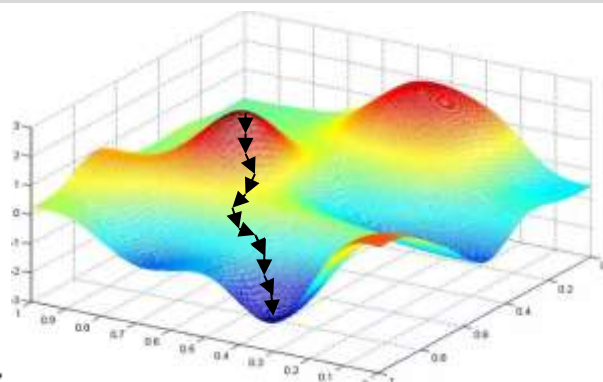
输入 $x = 0.6$, 权重 $w = 0.8$, 偏置 $b = 0.1$
目标输出 $target = 1.0$, 激活函数: $relu$

计算过程:

$$y = w * x + b = 0.8 * 0.6 + 0.1 = 0.58$$

$$z = \text{relu}(y) = 0.58$$

$$\text{Loss} = \text{target} - z = 0.42$$



2、后向传播（改错）

利用链式法则求权重梯度:

预测值 z 比 $target$ 小了 0.42 $z \uparrow 0.42$

y 影响了 z $y \uparrow 0.42$

x , w 和 b 影响了 y

如果 x 和 b 不变, $w \uparrow 0.7$ 系数 $x = 0.6$

如果 w 和 b 不变, $x \uparrow 0.525$ 系数 $w = 0.8$

最终梯度:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} * \frac{\partial z}{\partial y} * \frac{\partial y}{\partial w} = -1 * 1 * 0.6 = -0.6$$

结论：梯度为负，说明增加 w 可以减小 Loss!

课堂提问：后向传播梯度消失和爆炸

梯度参数	问题
w很大	
w很小	
x很大	
x很小	

$$z = \text{relu}(y_1 * w_{y1} + y_2 * w_{y2} + y_3 * w_{y3} + \cdots + b_z)$$

$$y = \text{relu}(x_1 * w_{x1} + x_2 * w_{x2} + x_3 * w_{x3} + \cdots + b_y)$$

课堂提问：后向传播梯度消失和爆炸

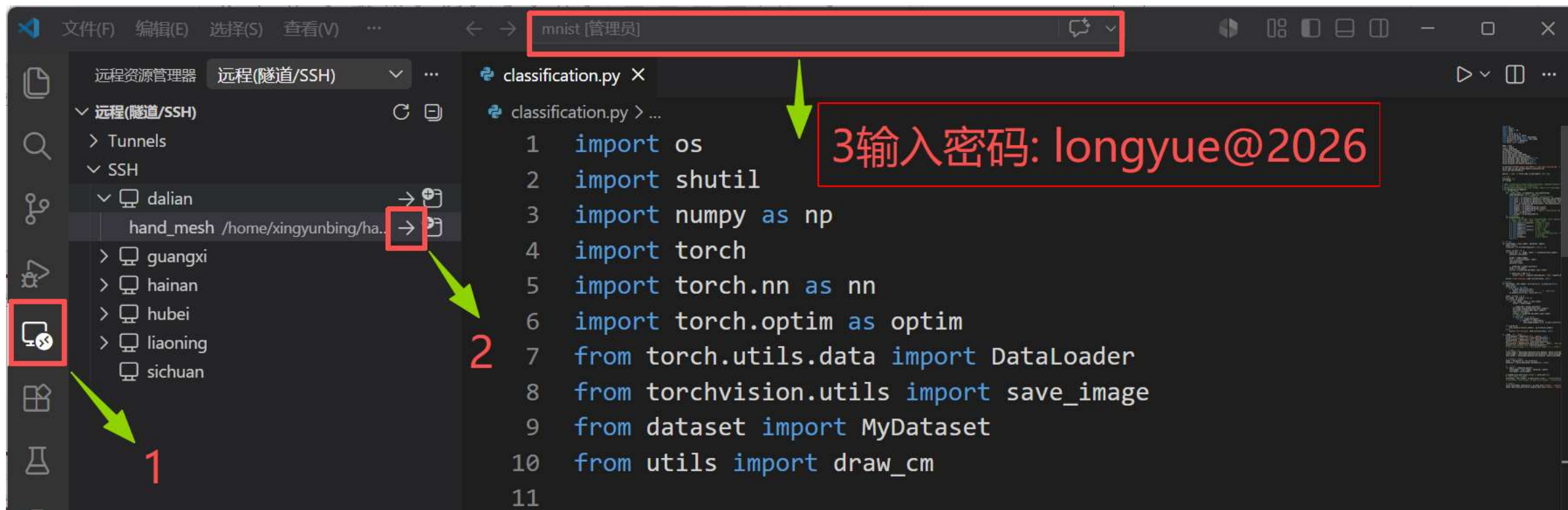
梯度参数	问题
w很大	x更新很小，后向传播消失
w很小	x更新很大，后向传播爆炸
x很大	w更新很小，贡献较小
x很小	w更新很大，BatchNorm2d

$$z = \text{relu}(y_1 * w_{y1} + y_2 * w_{y2} + y_3 * w_{y3} + \cdots + b_z)$$

$$y = \text{relu}(x_1 * w_{x1} + x_2 * w_{x2} + x_3 * w_{x3} + \cdots + b_y)$$

结论：w尽量分布在 ± 1 附近

pytorch实战：打开远程服务器



pytorch实战：构建识别器



Computational thinking and AI
计算思维与人工智能
兴趣班

```
37 class Recognizer(nn.Module):
38     # 定义不同的层
39     def __init__(self, in_channels=1, out_channels=10):
40         super(Recognizer, self).__init__()
41         # out_size = in_size + 2*padding - (kernel_size-1)
42         self.conv1 = nn.Conv2d(in_channels=in_channels, out_channels=8, kernel_size=5, padding=0) # 1x28x28 -> 8x24x24
43         self.conv2 = nn.Conv2d(in_channels=8, out_channels=16, kernel_size=5, padding=0) # 8x12x12 -> 16x8x8
44         self.conv3 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=0) # 16x4x4 -> 32x2x2
45         self.fc1 = nn.Linear(in_features=32*2*2, out_features=out_channels) # 32*2*2 -> 10
46         self.maxpool = nn.MaxPool2d(kernel_size=2)
47         self.avgpool = nn.AvgPool2d(kernel_size=2)
48         self.dropout = nn.Dropout(0.5) # 随机丢弃部分神经元，防止过拟合，不要用于浅层，可能会丢弃过多特征
49         self.relu = nn.ReLU(True)
50         self.softmax = nn.Softmax(dim=-1) # 输出概率分布
51         # conv1: 1*8*5*5=200, conv2: 8*16*5*5=3200, conv3: 16*32*3*3=4608, fc: 32*2*2*10=1280
52         # 参数量: 200 + 3200 + 4608 + 1280 = 9288
53         parameters_count = in_channels * 8 * 5 * 5 + \
54             8 * 16 * 5 * 5 + \
55             16 * 32 * 3 * 3 + \
56             32 * 2 * 2 * out_channels
57         print(f"Recognizer参数量: {parameters_count}")
58     # 使用定义的层
59     def forward(self, x):
60         x = self.relu(self.conv1(x)) # 28x28 -> 24x24
61         x = self.avgpool(x) # 24x24 -> 12x12
62         x = self.relu(self.conv2(x)) # 12x12 -> 8x8
63         x = self.avgpool(x) # 8x8 -> 4x4
64         x = self.relu(self.conv3(x)) # 4x4 -> 2x2
65         x = x.view(x.size(0), -1) # 32x2x2 -> 32*2*2
66         x = self.dropout(x)
67         x = self.fc1(x) # 32*2*2 -> 10
68         x = self.softmax(x)
69     return x
```

pytorch实战：混淆矩阵



实验结果

```
Epoch: [4/5], Step: [0/100], Loss: 1.6597
Epoch: [4/5], Step: [20/100], Loss: 1.6463
Epoch: [4/5], Step: [40/100], Loss: 1.6357
Epoch: [4/5], Step: [60/100], Loss: 1.5970
Epoch: [4/5], Step: [80/100], Loss: 1.6462
Train Accuracy: 82.36%
Valid Accuracy: 85.35%
```

```
09 00 10 10 00 00 00 00 21 00 0.180/50
45 00 00 03 00 00 00 00 02 00 0.000/50
41 00 01 02 00 05 00 00 01 00 0.020/50
40 00 07 00 00 00 00 00 03 00 0.000/50
28 00 08 13 00 00 00 00 01 00 0.000/50
20 00 25 04 00 00 00 00 01 00 0.000/50
29 00 04 17 00 00 00 00 00 00 0.000/50
31 00 15 00 00 00 00 00 04 00 0.000/50
40 00 04 04 00 01 00 00 01 00 0.020/50
36 00 09 01 00 00 01 00 03 00 0.000/50
0.028 0.000 0.012 0.000 0.000 0.000 0.000 0.000 0.027 0.000 0.022
```

训练数据



验证数据



测试数据



发现问题与原因

现象：模型在 MNIST 测试集准确率 >98%，但现场手写识别经常失败

根源：MNIST 数据集是“黑底白字”，而现场手写通常是“白底黑字”

计算机对颜色的反转敏感，它把反转视为不同特征

解决方案：数据增强

策略：训练时以 50% 概率互换前背景灰度

效果：模型学会识别形状而非颜色，对背景不敏感

结论：数据增强提升了真实场景下的鲁棒性

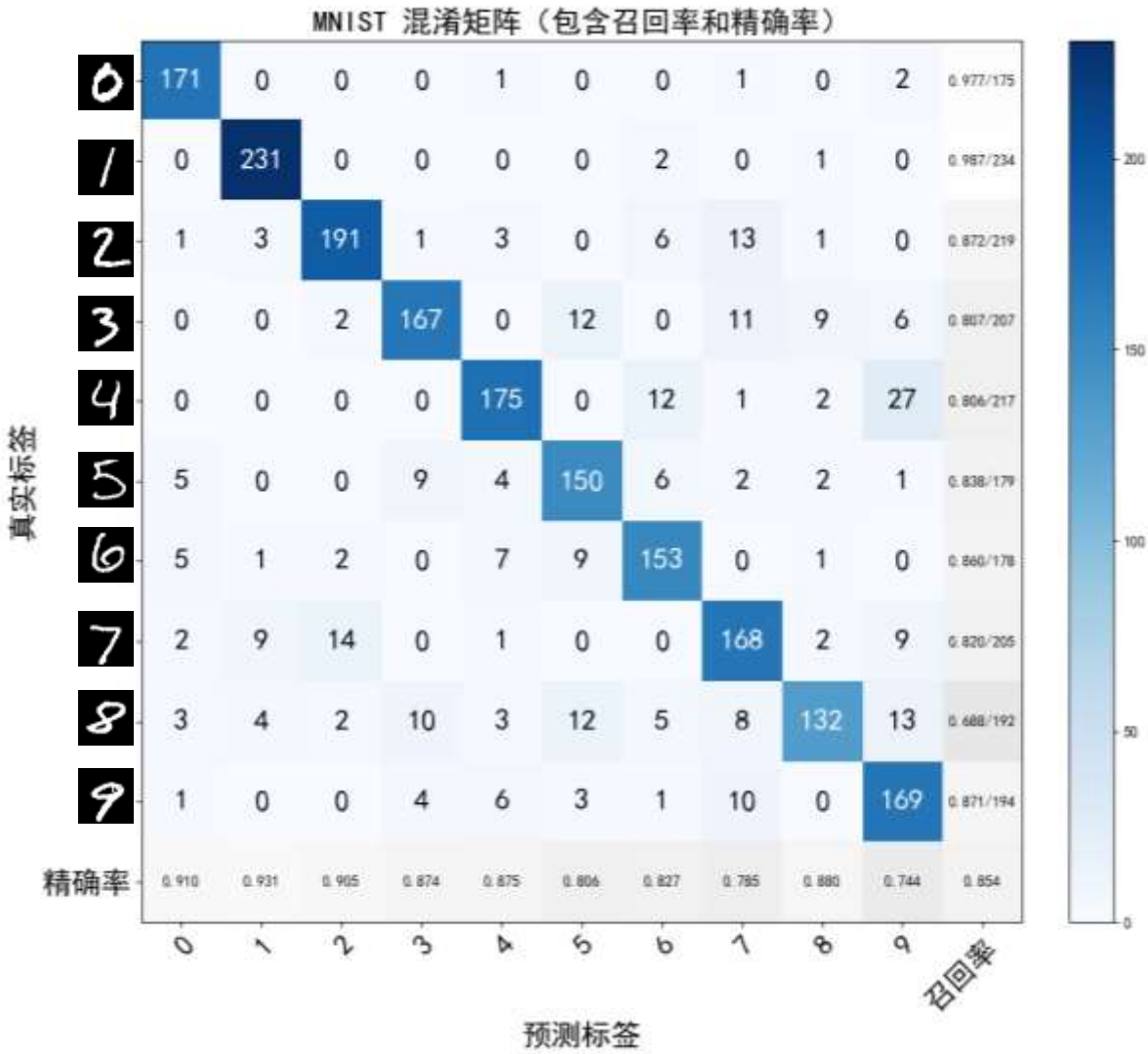
实验分析：颜色反转

```
125 train_dataset = MyDataset('data', device, True)
126 print(f'Train dataset size: {len(train_dataset)}')
127 valid_dataset = MyDataset('data', device, False)
128 print(f'Valid dataset size: {len(valid_dataset)}')
129 test_dataset = MyDataset(os.path.join('data', 'MNIST', 'test_images'), device)
130 print(f'Test dataset size: {len(test_dataset)}')
131 test_dataset.images = 1 - test_dataset.images # 反转测试集图像
```

pytorch实战：实验

```
33
34 # TODO: 修改这里的模型结构, 尝试不同的卷积层、池化层、全连接层等, 观察对性能的影响
35 # 目前卷积的in_channels和out_channels偏小
36 # 试试不同的卷积核尺寸kernel_size和不同的池化方式, 看看能不能提升性能
37 class Recognizer(nn.Module):
38     # 定义不同的层
39     def __init__(self, in_channels=1, out_channels=10):
40         super(Recognizer, self).__init__()
41         # out_size = in_size + 2*padding - (kernel_size-1)
42         self.conv1 = nn.Conv2d(in_channels=in_channels, out_channels=8, kernel_size=5, padding=0) # 1x28x28 -> 8x24x24
43         self.conv2 = nn.Conv2d(in_channels=8, out_channels=16, kernel_size=5, padding=0) # 8x12x12 -> 16x8x8
44         self.conv3 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=0) # 16x4x4 -> 32x2x2
45         self.fc1 = nn.Linear(in_features=32*2*2, out_features=out_channels) # 32*2*2 -> 10
46         self.maxpool = nn.MaxPool2d(kernel_size=2)
47         self.avgpool = nn.AvgPool2d(kernel_size=2)
48         self.dropout = nn.Dropout(0.5) # 随机丢弃部分神经元, 防止过拟合, 不要用于浅层, 可能会丢弃过多特征
49         self.relu = nn.ReLU(True)
50         self.softmax = nn.Softmax(dim=-1) # 输出概率分布
51         # conv1: 1*8*5*5=200, conv2: 8*16*5*5=3200, conv3: 16*32*3*3=4608, fc: 32*2*2*10=1280
52         # 参数量: 200 + 3200 + 4608 + 1280 = 9288
```

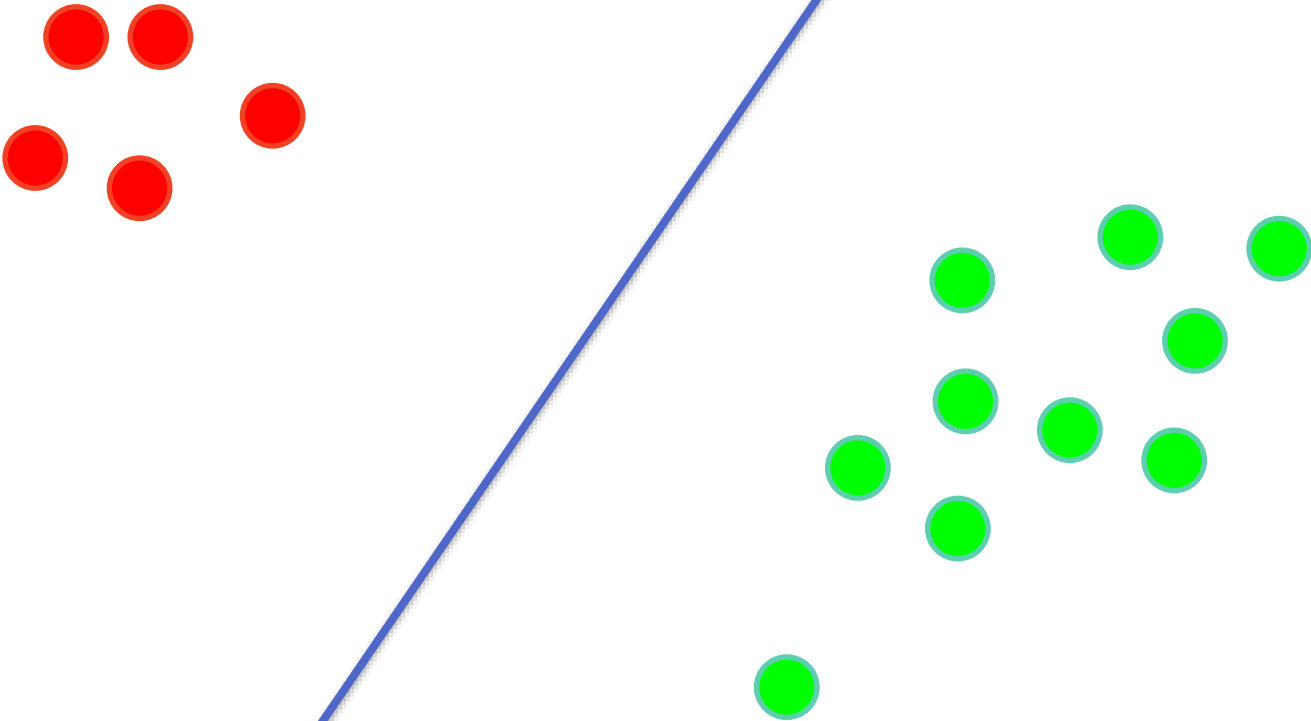

实验分析：混淆矩阵



卷积的动画演示



实验分析：非均衡



谢谢！ 请批评指正！